VRDL HW2 Report

112550011 李佑軒

April 21, 2026

1 Introduction

Github link: `https://github.com/Yu-Hsuan-1220/NYCU_Visual_Recognition_Using_Deep_Learning/tree/main/HW2`

The objective of this Lab is to develop a robust digit detection system capable of identifying and localizing numbers (0-9) in various image environments using DETR with ResNet50 backbone. The primary metric for success is the $mAP@[.5 : .95]$, which requires the model to not only find the digits (Recall) but also to provide highly accurate bounding box predictions (Precision).

Our core methodology integrates the efficient local attention mechanisms of Deformable DETR with a Two-stage query initialization technique to address the inherent challenges of detecting small-scale digits within complex, noisy backgrounds. By employing multi-scale feature sampling, the model achieves heightened sensitivity to fine-grained edge details, while the two-stage mechanism provides the decoder with precise spatial priors for more stable target localization. The optimization process follows a coarse-to-fine strategy, where heavy data augmentation is initially utilized to establish robustness against environmental artifacts, followed by a refinement phase that removes these augmentations and increases the coefficients for bounding box regression losses. This strategic transition compels the model to shift from generalized regional identification to pixel-level boundary alignment, successfully bridging the performance gap between robust classification and high-precision localization. Many code implementations are inspired by the official Deformable DETR codebase [3].

2 Method

2.1 Data Preprocessing

2.1.1 Augmentation Used

To bridge the domain gap between our training set and the challenging inference environment, we implemented a strategic data augmentation pipeline. Given that the test data frequently contains blurry, lowresolution, and inconsistently scaled digits, our preprocessing aims to improve the model's robustness and spatial invariance. The augmentations was carefully selected from pytorch documentation [5] and tested for effectiveness.

- **Scale and Crop (Resized_Fixed):** Our analysis indicated that the dataset is predominantly composed of small-scale objects, and the original image resolutions were often insufficient for the multiscale feature extraction requirements of the Deformable DETR backbone. To address this, we implemented a "scalethencrop" strategy. Images were first upscaled to a dimension slightly exceeding the target resolution and subsequently cropped to the final input size (e.g., 320×640). This approach effectively "zooms in" on the objects, forcing the model to prioritize localized, finegrained numerical features and mitigating its reliance on global image context, which often contains misleading background artifacts.

- **Atmospheric & Color Robustness:** To account for the "foggy" and blurred nature of the inference images, we applied **Gaussian Blur, Color Jitter, and Random Grayscale**. These simulate sensor noise and varying lighting conditions, ensuring the model remains discriminative even when color information is degraded or edges are indistinct.

- **Spatial Invariance via Translation:** We implemented **Random Translation** to artificially shift the position of digits within the frame. This prevents the model from developing a spatial bias and encourages it to search for digits across the entire image canvas.

- **Random Expand (Zoom-out Effect):** Since the inference dataset contains many instances where tiny digits are situated within large, complex backgrounds, we used an expansion technique. By placing the original image onto a larger, mean-filled canvas, we effectively simulated the distribution of small objects in expansive environments, improving the model's recall for distant or small-scale digits.

2.1.2 Augmentation Not Used

Certain standard augmentations were intentionally omitted to preserve the semantic and geometric integrity of the numerical data:

- **No Horizontal Flip:** This was strictly avoided because digit detection is semantically sensitive to orientation. Flipping can fundamentally change the identity of a digit (e.g., turning a 6 into a 9 or creating non-existent characters), which would introduce significant label noise.
- **No Rotation:** We decided against using rotation because it often results in "vague" bounding boxes. Since the detection framework requires axis-aligned bounding boxes, rotating a tilted digit creates a significantly larger box that encompasses excessive background noise. This destroys the precision of the dataset and hinders the model's ability to achieve high mAP_{75} scores.

2.2 Model Architecture

In this study, we transitioned from the standard DETR (DEtection TRansformer) to Deformable DETR and ultimately implemented a Two-stage Deformable DETR architecture. This progression was motivated by the specific challenges of digit detection: the high prevalence of small objects and the slow convergence rates associated with global self-attention mechanisms.

2.2.1 Deformable DETR

Standard DETR [1] utilizes a global self-attention mechanism, which calculates the relationship between every pair of pixels. This leads to a computational complexity of $O(N^2)$ (where N is the number of pixels), making it extremely slow to train on high-resolution images. Furthermore, the global attention tends to "dilute" the signal for small objects, as the model struggles to attend to localized features amidst the vast background.

Deformable DETR [7, 4] addresses these limitations by introducing the Multi-Scale Deformable Attention (MS-DeformAttn) module. Instead of attending to all pixels, each query attends only to a small, learnable set of sampling points ($k = 4$ in our implementation) around a reference point across multiple feature scales.

- **Efficiency:** The complexity is reduced to $O(N)$, significantly accelerating training convergence.
- **Small Object Performance:** By utilizing multi-scale feature maps (C3, C4, C5 in our implementation), the model can leverage higher-resolution maps (Stride 8) to capture the fine-grained details of tiny digits that are often lost in deeper, lower-resolution layers.

We utilized a ResNet-50 backbone with Frozen Batch Normalization to extract multi-scale features. To better handle small digits, we extracted three scales of feature maps with strides of 8, 16, and 32.

- C3 (Stride 8): Provides the necessary spatial resolution for localized edge detection.
- C4 & C5 (Strides 16/32): Provide the semantic context required for character classification.
- The features are projected into a hidden dimension of 256 and augmented with 2-D sine positional encodings before being fed into the transformer.

We enabled Iterative Box Refinement, where each decoder layer predicts a residual offset to the bounding box predicted by the previous layer. This allows the model to progressively "tighten" the box around the digit, which is crucial for achieving high scores in the mAP_{75} metric where pixel-level precision is required.

2.2.2 Two-stage Deformable DETR

The most significant architectural enhancement was the implementation of the Two-stage mechanism. However, the final performance of these two architectures is roughly the same in my experiments on this task.

In the One-stage version, the decoder uses 100 static, learnable query embeddings. These queries are independent of the image content and must learn where to look through thousands of iterations.

In our Two-stage implementation:

- **Encoder Proposal Stage:** The transformer encoder processes the multi-scale features and predicts "objectness" scores and preliminary bounding boxes for every feature pixel.
- **Top-K Selection:** We select the top 100 highest-scoring pixels as Proposals.
- **Decoder Initialization:** Instead of using static embeddings, these 100 proposals are used to initialize the Reference Points and Content Queries for the decoder.

2.3 Criterion for Training

The core of our training logic lies in the Set-based Loss, which treats the model's predictions and the ground truth (GT) as two sets and finds the optimal "one-to-one" assignment between them.

In conventional object detectors (like YOLO), multiple anchors can be assigned to a single object, requiring NMS to filter duplicates. In our implementation, we utilize the Hungarian Algorithm (via linear_sum_assignment) to perform Bipartite Matching. [2]

This ensures that each ground truth digit is assigned to exactly one specific query. This one-to-one mapping is what allows Deformable DETR to be "NMS-free"—the model learns to suppress duplicate predictions during training because only the best-matching query receives a "positive" gradient.

To find the optimal assignment, we compute a Cost Matrix C where each entry $C_{i,j}$ represents the "misalignment" between prediction i and ground truth j:

$$C = \lambda_{cls} \cdot C_{prob} + \lambda_{L1} \cdot C_{bbox} + \lambda_{giou} \cdot C_{giou}$$

- **Classification Cost (C_{prob}):** We use the Softmax probability of the correct class. A query that identifies the digit correctly receives a lower cost.
- **L1 Bounding Box Cost (C_{bbox}):** Measures the absolute difference between the predicted and GT coordinates (cx, cy, w, h).
- **Generalized IoU Cost (C_{giou})[6]:** Since digits can be very small and may not overlap initially, standard IoU would provide zero gradients. GIoU ensures the model still receives a signal even when the boxes are far apart, encouraging them to move toward each other.

We calculate the loss only for the matched pairs (where $C_{i,j}$ is minimized) and treat unmatched predictions as negatives (background class). This encourages the model to learn precise localization and classification simultaneously, which is essential for achieving high mAP scores, especially at stricter IoU thresholds.

2.3.1 Auxiliary Loss

Digit detection, especially for blurry and small targets, is a complex optimization problem. To stabilize and accelerate training, we utilized Auxiliary Loss (aux_loss).

The Deformable DETR decoder consists of multiple layers (6 in our configuration). While only the final layer's output is used during inference, aux_loss calculates the Bipartite Matching and Set Criterion for every intermediate decoder layer.

This serves as Deep Supervision, providing a training signal to early layers. It forces the model to refine its predictions layer-by-layer rather than relying solely on the final output. We found this crucial for the "Iterative Box Refinement" mechanism, as it ensures that the "offsets" being predicted are based on reasonably accurate intermediate coordinates.

2.4 Inference

The inference procedure was designed to maximize recall while maintaining high localization precision. Unlike traditional detectors that rely heavily on dense anchor filtering, our Deformable DETR pipeline utilizes a streamlined decoding process optimized for the sparse set of object queries.

For each test image, the model outputs 100 object queries, each consisting of a class logit vector and a bounding box coordinate. Our decoding strategy follows these steps:

- **Background Exclusion:** We explicitly remove the "no-object" (background) class during the Softmax or Sigmoid calculation. By focusing only on the $K = 10$ digit classes, we ensure that the model's confidence is distributed purely among the target categories.
- **Top-1 per Query Selection:** For each individual query, we select only the class with the highest probability (Top-1). This prevents a single query from predicting multiple overlapping labels for the same region, which simplifies the output and ensures each query acts as a unique object candidate.

We applied a relatively low score threshold of 0.01 to filter the predictions.

- **The Benefit of a Low Threshold:** In the context of the mAP (mean Average Precision) metric, maintaining a high recall is critical. A threshold of 0.01 allows the model to retain low-confidence detections that might otherwise be discarded. For our task, where many digits are blurry or obscured by background "cracks," this ensures that potential candidates are preserved for the final evaluation, significantly boosting the overall recall rate.

2.5 Hyperparameter Settings and Training Details

Our training process was divided into two distinct phases to balance the trade-off between model robustness and localization precision. We utilized the AdamW optimizer with a Cosine Annealing learning rate scheduler.

2.5.1 Phase I: Robust Feature Exploration

In the first phase, the model was trained for 100 epochs from scratch. The primary goal was to encourage the model to learn generalized digit features while ignoring high-frequency background noise.

- **Learning Rate Strategy:** We employed a base learning rate of 1×10^{-4} with a 10-epoch warmup period. The backbone learning rate was set to a lower factor of 1×10^{-5} to preserve pretrained spatial features.
- **Heavy Augmentation:** To prevent overfitting on the specific textures of the training images (e.g., wall cracks), we applied an aggressive augmentation suite including Random Translation, ISO Noise, and Random Expand (max ratio 0.8). This forced the model to localize digits regardless of their position or sensor noise.
- **Backbone Configuration:** We set freeze_at=0 to allow the entire ResNet-50 backbone to adapt to the low-quality digit images.

2.5.2 Phase II: High-Precision Refinement (Fine-tuning)

After the model plateaued (around epoch 14), we initiated a refinement phase. This stage focused on "snapping" the bounding boxes to the exact boundaries of the digits to maximize the mAP_{75} score.

- **Clean-tuning:** We removed the heavy augmentations to provide the model with "clean" images, allowing it to focus on pixel-level boundary alignment.
- **Backbone Stabilization:** We increased freeze_at to 2, freezing the shallower layers of the ResNet-50 to stabilize the feature extraction layers that had already converged.
- **Loss Weight Re-scaling:** We significantly increased the coefficients for localization losses. Specifically, loss_bbox_coef was raised from 5.0 to 8.0, and loss_giou_coef from 2.0 to 5.0. This prioritized the Geometric IoU and L_1 distance, forcing the queries to minimize even the smallest spatial offsets.
- **Optimization Precision:** The learning rate was reduced by 20x (to 5×10^{-6}) to perform surgical updates to the weights without disrupting the learned classification categories.

2.5.3 Hyperparameter Settings For Two Phases

Table 1: Hyperparameter Settings for Phase I and Phase II

	Phase I	**Phase II**
Encoder Layers	6	6
Decoder Layers	6	6
Attention Heads	8	8
Object Queries	100	100
Hidden Dimension	256	256
Feedforward Dim	1,024	1,024
Feature Levels	3 (C3, C4, C5)	3 (C3, C4, C5)
Dropout	0.1	0.1
Optimizer	AdamW	AdamW
Base Learning Rate	1×10^{-4}	5×10^{-6}
Backbone LR	1×10^{-5}	5×10^{-7}
Weight Decay	1×10^{-4}	1×10^{-3}
Batch Size (Effective)	16 (Total 32 via grad-acc)	16 (Total 32 via grad-acc)
BBox Loss Coef (λ_{L1})	5.0	8.0
GIoU Loss Coef (λ_{giou})	2.0	5.0
Warmup Epochs	10	0
Backbone Freeze Stage	0 (None)	2 (Layer 0, 1, 2)
Augmentations	Translation, Expand, ISO Noise	Disabled (Clean-tuning)

3 Results

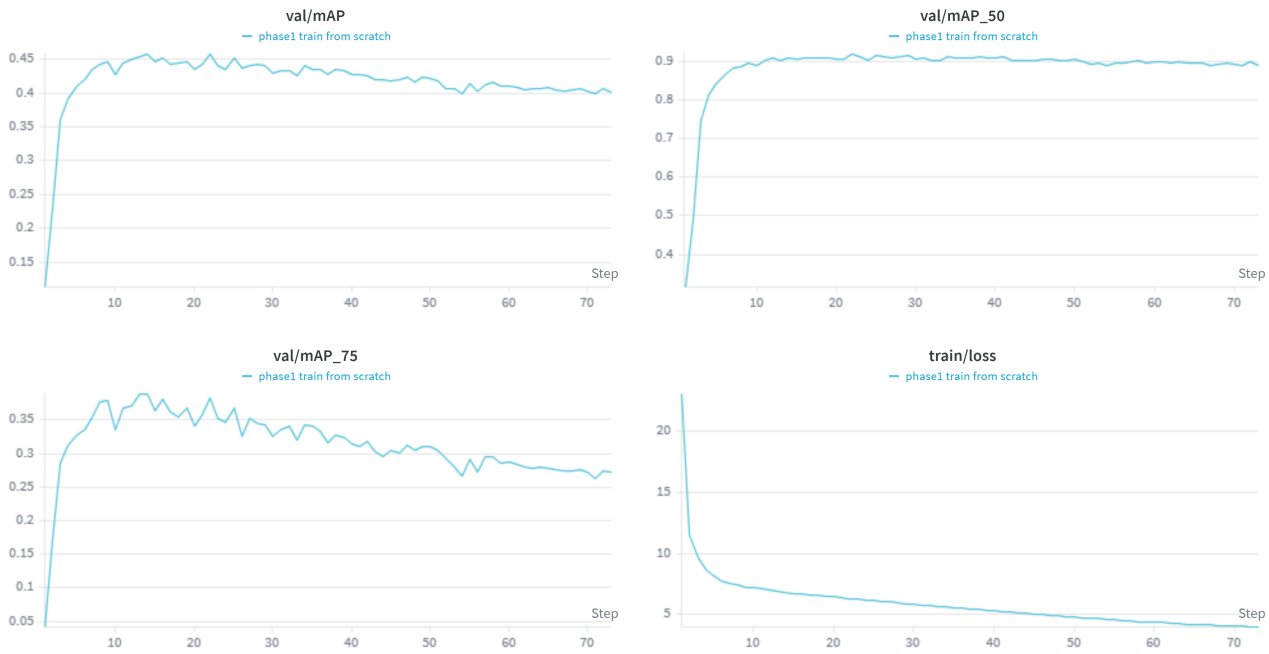

Figure 1: Phase I Training Curve

The performance of the first training phase (from scratch with heavy augmentation) provided critical insights into the model's behavior. As shown in Figure 1, several key trends emerged during the 100-epoch exploration phase:

A significant disparity was observed between the recognition capability and localization precision.

- mAP_{50} **Performance:** The model achieved an impressive mAP_{50} of approximately 0.9 very early in the training process. This indicates that the Deformable DETR is highly effective at correctly identifying and categorizing digits, even under heavy data augmentation.
- mAP_{75} **Performance:** In contrast, the mAP_{75} (which requires a much higher Intersection-over-Union threshold) peaked at only 0.38. This gap suggests that while the model successfully localizes the "general area" of the digits, it struggles with pixel-level boundary alignment, likely due to the noise and blur introduced during the augmentation process.

The validation curves also exhibit a classic overfitting pattern as the training progressed:

- **Training Loss vs. Validation Metrics:** While the Training Loss continued to decrease steadily (dropping from >20 to 4), the validation scores—specifically val/mAP and val/mAP_75—peaked around step 15 and then began a consistent decline.
- **Metric Degradation:** The mAP_{75} score dropped from its peak of 0.38 toward 0.27 in the later stages. This confirms that the model started to over-optimize for the specific noise textures and background artifacts (such as wall cracks) present in the training set, leading to a loss of generalization on the validation set.

Figure 2: Phase II Training Curve

The results from Phase I indicated that continuing the initial training regime would only lead to further overfitting without improving localization precision. The primary bottleneck was identified as the trade-off between the robustness provided by heavy augmentations and the precision required for high IoU scores.

To bridge this 0.5 mAP gap, a second phase of "Clean-tuning" was mandated. By loading the best-performing weights from Phase I (around epoch 14), removing the stochastic noise of augmentations, and re-scaling the L_1 and $GIoU$ loss coefficients, we aimed to "snap" the existing query predictions onto the ground truth boundaries, transforming the high mAP_{50} recall into true mAP_{75} precision.

As shown in Figure 2, after using the new training strategy and train for one epoch, mAP_75 significantly increased from 0.38 to 0.42, which is a remarkable improvement in just one epoch. And the mAP reach the new peak of 0.4746 at epoch 2, this demonstrates that the model successfully refined its bounding box predictions to better align with the ground truth, confirming the effectiveness of the fine-tuning phase in enhancing localization precision.

19	yu_hsuan_1220	2026-04-18 23:47	686170	112550011	0.39

Figure 3: Leaderboard Results

After adding all the improvements such as augmentations, our final model achieved a Validation mAP of 0.47 and a Test mAP of 0.39.

4 Additional Experiments

To further improve the model's performance beyond the baseline Deformable DETR, we conducted a series of structural and methodological experiments. These were driven by specific hypotheses regarding the unique challenges of the blurry digit dataset.

4.1 Structural Improvement

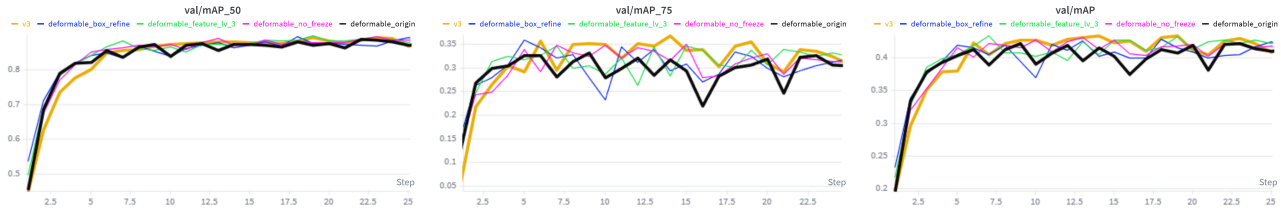

Figure 4: Experiment of Deformable DETR

From figure 4, we experiment the Deformable DETR with and without box refine, with different input feature levels to transformer, and freeze different backbone layers when training.

Hypothesis and Methodology:

- **Full gradient flow (freeze_at=0):** The lower layers of a ResNet-50 pretrained on ImageNet capture generic textures. However, our dataset contains specific artifacts (blurs and wall cracks) that are drastically different from ImageNet. We hypothesized that freezing the first two layers (freeze_at=2) would limit the model's ability to extract domain-specific features. We conducted an experiment comparing freeze_at=2 against freeze_at=0 (unfreezing the entire backbone).

- **Multi-scale feature levels:** Deformable DETR typically uses four feature levels (C3, C4, C5, and a generated C6). However, for extremely small digits, a stride-64 feature map (C6) provides a resolution so low that a single pixel covers a vast area, likely introducing more background noise than useful semantic information. We restricted the multi-scale input to only C3, C4, and C5 (strides 8, 16, and 32), effectively removing the C6 level.

- **Iterative Box Refinement:** Standard one-step bounding box regression is insufficient for small, blurry digits where the boundary is indistinct. We hypothesized that a multi-step "coarse-to-fine" approach would allow the model to converge more accurately on high-IoU targets. We enabled the with_box_refine module, allowing each decoder layer to predict a residual offset based on the reference points provided by the previous layer.

Results: Three of the hypotheses were validated by the experiment. The mAP are more consistent and higher. The black line is the original setting of Deformable DETR, and the yellow line is the new setting with all three improvements.

4.2 Augmentation Improvement

Figure 5: Ablation Study of Gaussian Blur Augmentation

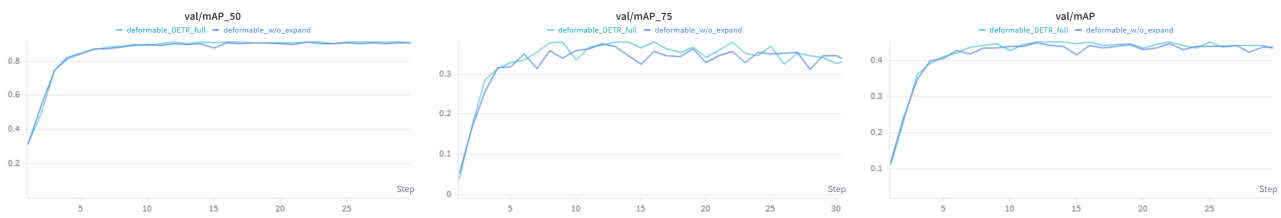

Figure 6: Ablation Study of Expand Augmentation

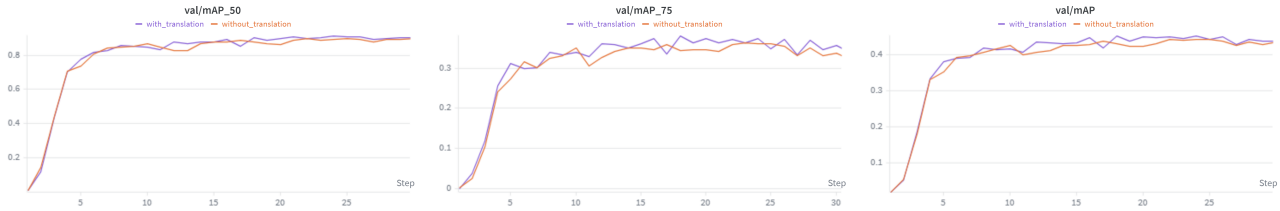

Figure 7: Ablation Study of Translation Augmentation

The reason why I use these augmentations are mentioned in the data preprocessing section. From figure 5, 6 and 7, we can see that all three augmentations (Gaussian Blur, Random Expand and Random Translation) contribute to the improvement of the model's performance.

A key highlight of our experimental results is the significant improvement in the model's generalization capability. While the introduction of advanced augmentations led to a higher absolute performance, its most critical impact was the reduction of the "generalization gap" between validation and test environments.

Table 2: Gap between Validation and Test mAP with and without Augmentation

	Validation	**Test**
Without Augmentation	0.43	0.33
With all Augmentation	0.47	0.39

This results in a performance discrepancy of only 0.08, which is a notable improvement compared to our earlier baseline models where the gap consistently exceeded 0.10. The narrowing of this gap indicates that the model is no longer merely memorizing the training/validation distribution. Instead, the heavy use of spatial and noise augmentations has forced the model to learn invariant numerical features. This enhanced robustness allows the model to better handle the domain shift and unpredictable artifacts present in the test set, ensuring reliable performance across different image qualities.

4.3 Two-stage Deformable DETR

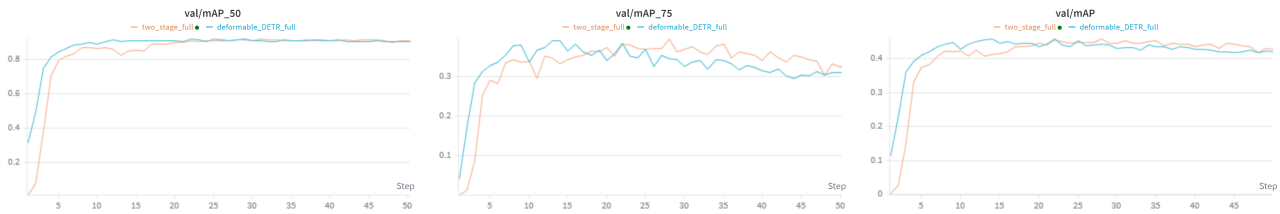

Figure 8: Comparison of Two-stage Deformable DETR

As shown in Figure 8, the Two-stage variant exhibits superior stability and final performance. The most notable difference is observed in the mAP_{75} and total mAP curves.

- **Standard Deformable DETR (Blue Line):** While it converges faster in the very early steps, it exhibits significant fluctuations and a noticeable downward trend after approximately Step 20. This suggests that the model's static queries are susceptible to "distraction" by background artifacts, leading to unstable localization.
- **Two-stage Deformable DETR (Orange Line):** The training trajectory is considerably smoother and maintains a higher performance plateau. This stability is attributed to the Encoder-proposal mechanism, which filters out the majority of background noise before the decoder even begins its task. By starting with high-quality, image-dependent proposals, the optimization landscape for the decoder becomes much more well-behaved.
- While both models reach a similar ceiling in mAP_{50} (0.9), the Two-stage model consistently outperforms the one-stage baseline in mAP_{75}.

However, I cannot find out a way to further finetune the Two-stage model to achieve higher mAP. So the final submission is still based on the One-stage Deformable DETR, which has a slightly higher mAP after finetuning than the Two-stage version in my experiments.

References

[1] Nicolas Carion et al. "End-to-End Object Detection with Transformers". In: *arXiv preprint arXiv:2005.12872* (2020).

[2] jackroos. *Criterion for deformable DETR GitHub*. GitHub. Nov. 2020. URL: https://github.com/fundamentalvision/Deformable-DETR/blob/main/models/deformable_detr.py#L198 (visited on 04/20/2026).

[3] jackroos. *Deformable DETR GitHub Repository*. GitHub. Jan. 2020. URL: https://github.com/fundamentalvision/Deformable-DETR/tree/main (visited on 04/20/2026).

[4] jackroos. *Deformable-DETR GitHub Repository*. Hazy Research Blog. Jan. 2022. URL: https://github.com/fundamentalvision/Deformable-DETR (visited on 04/20/2026).

[5] PyTorch Team. *PyTorch Vision Transforms Documentation*. PyTorch Documentation. Jan. 2020. URL: https://docs.pytorch.org/vision/0.22/transforms.html (visited on 04/20/2026).

[6] Stanford University. *Giou explanation*. Stanford University. June 2019. URL: https://giou.stanford.edu/ (visited on 04/20/2026).

[7] Xizhou Zhu et al. "Deformable DETR: Deformable Transformers for End-to-End Object Detection". In: *arXiv preprint arXiv:2010.04159* (2020).